Práctica de modelos lineales generalizados: demanda de bicicletas en Seúl

25/07/2026

1. Contexto del problema

Los sistemas de bicicletas compartidas deben anticipar la demanda para disponer de suficientes bicicletas en los momentos de mayor uso. En esta práctica se analiza el número de bicicletas arrendadas por hora en Seúl, junto con variables meteorológicas y de calendario.

La actividad adapta a **R** la lógica del tutorial de modelos lineales generalizados de *Transparent ML*, que compara la regresión lineal con la regresión de Poisson para estudiar datos de conteo:

- Tutorial de referencia: https://pykale.github.io/transparentML/08-glm-svm/glm.html (https://pykale.github.io/transparentML/08-glm-svm/glm.html)
- Fuente original de los datos: https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand (https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand)

Objetivos

Al finalizar la práctica se espera que el estudiante pueda:

1. reconocer una variable respuesta de conteo;
2. explicar por qué una regresión lineal puede ser inadecuada para conteos;
3. ajustar e interpretar un modelo de regresión de Poisson;
4. evaluar la presencia de sobredispersión;
5. ajustar un modelo quasi-Poisson como alternativa inferencial;
6. construir una clasificación binaria sencilla mediante regresión logística;
7. evaluar un clasificador mediante una matriz de confusión, métricas y curva ROC.

2. Paquetes necesarios

Antes de ejecutar el documento, instale una sola vez los paquetes que no tenga disponibles:

```
install.packages(c(
  "tidyverse",
  "lubridate",
  "broom",
  "pROC",
  "knitr"
))
```

Cargamos los paquetes utilizados en el análisis.

```
library(tidyverse)
library(lubridate)
library(broom)
library(knitr)
```

3. Importación de la base de datos

El archivo `SeoulBikeData.csv` debe encontrarse en la misma carpeta que este documento `.Rmd`.

La codificación `latin1` es necesaria porque algunos nombres de variables contienen símbolos como grados Celsius.

```
archivo <- "SeoulBikeData.csv"

if (!file.exists(archivo)) {
  stop(
    "No se encontró el archivo SeoulBikeData.csv. ",
    "Guarde el archivo CSV en la misma carpeta del documento RMarkdown."
  )
}

bike_raw <- read.csv(
  file = archivo,
  fileEncoding = "latin1",
  check.names = FALSE,
  stringsAsFactors = FALSE
)

dim(bike_raw)
```

```
## [1] 8760   14
```

```
head(bike_raw, 4)
```

```
##         Date Rented Bike Count Hour Temperature(°C) Humidity(%)
## 1 01/12/2017               254    0            -5.2          37
## 2 01/12/2017               204    1            -5.5          38
## 3 01/12/2017               173    2            -6.0          39
## 4 01/12/2017               107    3            -6.2          40
##   Wind speed (m/s) Visibility (10m) Dew point temperature(°C)
## 1              2.2             2000                      -17.6
## 2              0.8             2000                      -17.6
## 3              1.0             2000                      -17.7
## 4              0.9             2000                      -17.6
##   Solar Radiation (MJ/m2) Rainfall(mm) Snowfall (cm) Seasons    Holiday
## 1                       0            0             0  Winter No Holiday
## 2                       0            0             0  Winter No Holiday
## 3                       0            0             0  Winter No Holiday
## 4                       0            0             0  Winter No Holiday
##   Functioning Day
## 1             Yes
## 2             Yes
## 3             Yes
## 4             Yes
```

La base contiene una observación por cada combinación de fecha y hora. La variable respuesta es el número de bicicletas arrendadas durante esa hora.

4. Preparación de los datos

Primero reemplazamos los nombres originales por nombres breves en español. Luego convertimos la fecha y las variables categóricas al formato adecuado.

```
names(bike_raw) <- c(
  "fecha",
  "bicicletas",
  "hora",
  "temperatura",
  "humedad",
  "velocidad_viento",
  "visibilidad",
  "punto_rocio",
  "radiacion_solar",
  "lluvia",
  "nieve",
  "estacion",
  "feriado",
  "funcionamiento"
)

meses_es <- c(
  "Ene", "Feb", "Mar", "Abr", "May", "Jun",
  "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"
)

bike <- bike_raw |>
  mutate(
    fecha = dmy(fecha),
    estacion = trimws(estacion),
    feriado = trimws(feriado),
    funcionamiento = trimws(funcionamiento),
    estacion = factor(
      estacion,
      levels = c("Winter", "Spring", "Summer", "Autumn"),
      labels = c("Invierno", "Primavera", "Verano", "Otoño")
    ),
    feriado = factor(
      feriado,
      levels = c("No Holiday", "Holiday"),
      labels = c("No", "Sí")
    ),
    funcionamiento = factor(
      funcionamiento,
      levels = c("No", "Yes"),
      labels = c("No", "Sí")
    ),
    mes_num = month(fecha),
    mes = factor(mes_num, levels = 1:12, labels = meses_es),
    hora_f = factor(hora, levels = 0:23),
    dia_semana = wday(fecha, week_start = 1),
    dia_laboral = factor(
      if_else(dia_semana <= 5 & feriado == "No", "Sí", "No"),
      levels = c("No", "Sí")
    ),
    # Representaciones cíclicas útiles para clasificación y predicción temporal.
```

```
    hora_sen = sin(2 * pi * hora / 24),
    hora_cos = cos(2 * pi * hora / 24),
    mes_sen = sin(2 * pi * mes_num / 12),
    mes_cos = cos(2 * pi * mes_num / 12)
  ) |>
  arrange(fecha, hora)

glimpse(bike)
```

```
## Rows: 8,760
## Columns: 23
## $ fecha            <date> 2017-12-01, 2017-12-01, 2017-12-01, 2017-12-01, 2017…
## $ bicicletas       <int> 254, 204, 173, 107, 78, 100, 181, 460, 930, 490, 339,…
## $ hora             <int> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,…
## $ temperatura      <dbl> -5.2, -5.5, -6.0, -6.2, -6.0, -6.4, -6.6, -7.4, -7.6,…
## $ humedad          <int> 37, 38, 39, 40, 36, 37, 35, 38, 37, 27, 24, 21, 23, 2…
## $ velocidad_viento <dbl> 2.2, 0.8, 1.0, 0.9, 2.3, 1.5, 1.3, 0.9, 1.1, 0.5, 1.2…
## $ visibilidad      <int> 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000, 2000,…
## $ punto_rocio      <dbl> -17.6, -17.6, -17.7, -17.6, -18.6, -18.7, -19.5, -19.…
## $ radiacion_solar  <dbl> 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.01,…
## $ lluvia           <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,…
## $ nieve            <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,…
## $ estacion         <fct> Invierno, Invierno, Invierno, Invierno, Invierno, Inv…
## $ feriado          <fct> No, No, No, No, No, No, No, No, No, No, No, No, No, N…
## $ funcionamiento   <fct> Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, S…
## $ mes_num          <dbl> 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 1…
## $ mes              <fct> Dic, Dic, Dic, Dic, Dic, Dic, Dic, Dic, Dic, Dic, Dic…
## $ hora_f           <fct> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,…
## $ dia_semana       <dbl> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,…
## $ dia_laboral      <fct> Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, Sí, S…
## $ hora_sen         <dbl> 0.00000000000000000000, 0.258819045102520739476 4, 0…
## $ hora_cos         <dbl> 1.00000000000000000000, 0.965925826289068312213 71,…
## $ mes_sen          <dbl> -0.00000000000000024492 94, -0.00000000000000024492 94,…
## $ mes_cos          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,…
```

4.1. Diccionario resumido de variables

```
diccionario <- tribble(
  ~variable, ~descripcion,
  "fecha", "Fecha de la observación",
  "bicicletas", "Número de bicicletas arrendadas durante la hora",
  "hora", "Hora del día, entre 0 y 23",
  "temperatura", "Temperatura en grados Celsius",
  "humedad", "Humedad relativa, en porcentaje",
  "velocidad_viento", "Velocidad del viento en metros por segundo",
  "visibilidad", "Visibilidad medida en unidades de 10 metros",
  "punto_rocio", "Temperatura del punto de rocío",
  "radiacion_solar", "Radiación solar en MJ/m²",
  "lluvia", "Lluvia en milímetros",
  "nieve", "Nieve en centímetros",
  "estacion", "Estación del año",
  "feriado", "Indicador de día feriado",
  "funcionamiento", "Indica si el sistema estaba funcionando"
)

kable(diccionario, align = c("l", "l"))
```

variable	descripcion
fecha	Fecha de la observación
bicicletas	Número de bicicletas arrendadas durante la hora
hora	Hora del día, entre 0 y 23
temperatura	Temperatura en grados Celsius
humedad	Humedad relativa, en porcentaje
velocidad_viento	Velocidad del viento en metros por segundo
visibilidad	Visibilidad medida en unidades de 10 metros
punto_rocio	Temperatura del punto de rocío
radiacion_solar	Radiación solar en MJ/m²
lluvia	Lluvia en milímetros
nieve	Nieve en centímetros
estacion	Estación del año
feriado	Indicador de día feriado
funcionamiento	Indica si el sistema estaba funcionando

5. Exploración inicial

5.1. Valores faltantes

```
faltantes <- bike |>
  summarise(across(everything(), ~ sum(is.na(.x)))) |>
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "n_faltantes"
  )

kable(faltantes)
```

variable	n_faltantes
fecha	0
bicicletas	0
hora	0
temperatura	0
humedad	0
velocidad_viento	0
visibilidad	0
punto_rocio	0
radiacion_solar	0
lluvia	0
nieve	0
estacion	0
feriado	0
funcionamiento	0
mes_num	0
mes	0
hora_f	0
dia_semana	0
dia_laboral	0
hora_sen	0
hora_cos	0
mes_sen	0

variable	n_faltantes
mes_cos	0

5.2. Resumen de la variable respuesta

```
summary(bike$bicicletas)
```

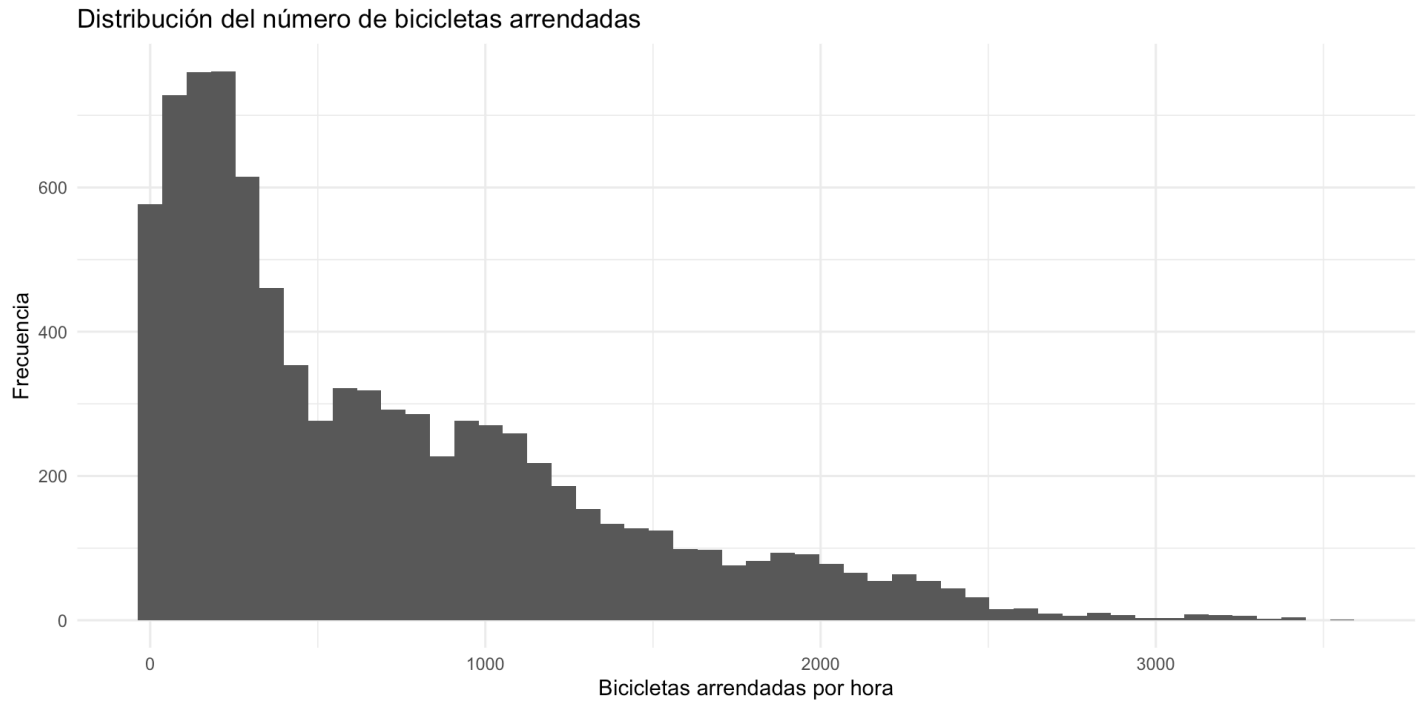
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.0   191.0   504.5   704.6  1065.2  3556.0
```

```
bike |>
  summarise(
    n = n(),
    media = mean(bicicletas),
    varianza = var(bicicletas),
    desviacion = sd(bicicletas),
    minimo = min(bicicletas),
    maximo = max(bicicletas)
  ) |>
  kable(digits = 2)
```

n	media	varianza	desviacion	minimo	maximo
8760	704.6	416021.7	645	0	3556

Una variable de conteo toma valores enteros no negativos. En este caso, el número de bicicletas puede ser 0, 1, 2, y así sucesivamente.

```
ggplot(bike, aes(x = bicicletas)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribución del número de bicicletas arrendadas",
    x = "Bicicletas arrendadas por hora",
    y = "Frecuencia"
  ) +
  theme_minimal()
```



5.3. Días sin funcionamiento

Antes de modelar, verificamos qué ocurre cuando el sistema aparece como no operativo.

```
bike |>
  group_by(funcionamiento) |>
  summarise(
    observaciones = n(),
    media = mean(bicicletas),
    minimo = min(bicicletas),
    maximo = max(bicicletas),
    .groups = "drop"
  ) |>
  kable(digits = 2)
```

funcionamiento	observaciones	media	minimo	maximo
No	295	0.00	0	0
Sí	8465	729.16	2	3556

Los ceros registrados en días sin funcionamiento no representan una demanda naturalmente baja: representan horas en las que el servicio estaba cerrado. Por esta razón, para modelar la **demanda observada cuando el sistema opera**, trabajaremos únicamente con las horas de funcionamiento.

```
datos_operativos <- bike |>
  filter(funcionamiento == "Sí") |>
  droplevels()

nrow(datos_operativos)
```

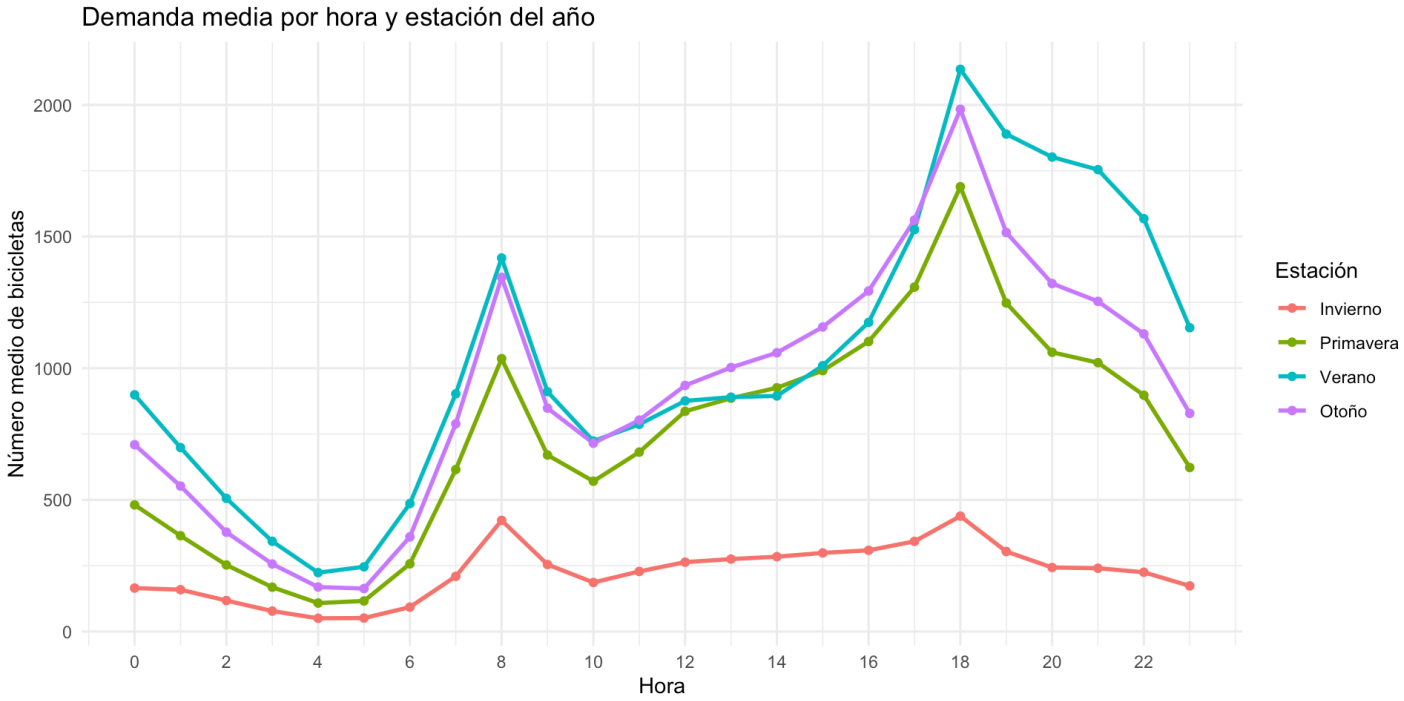
```
## [1] 8465
```

```
range(datos_operativos$bicicletas)
```

```
## [1]    2 3556
```

5.4. Demanda media según hora y estación

```
datos_operativos |>
  group_by(estacion, hora) |>
  summarise(
    demanda_media = mean(bicicletas, na.rm = TRUE),
    .groups = "drop"
  ) |>
  ggplot(
    aes(
      x = hora,
      y = demanda_media,
      group = estacion,
      color = estacion
    )
  ) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.6) +
  scale_x_continuous(breaks = seq(0, 23, by = 2)) +
  labs(
    title = "Demanda media por hora y estación del año",
    x = "Hora",
    y = "Número medio de bicicletas",
    color = "Estación"
  ) +
  theme_minimal()
```



Este gráfico permite identificar patrones horarios. En una ciudad, la hora no debe interpretarse como una variable lineal ordinaria: la diferencia entre las 23:00 y las 00:00 no es comparable con una separación de 23 horas. En la primera parte la trataremos como factor, siguiendo el enfoque del tutorial de referencia.

6. Regresión lineal como modelo inicial

Aunque la respuesta es un conteo, comenzamos con una regresión lineal para observar sus limitaciones.

El modelo utiliza:

- mes del año;
- hora del día como variable categórica;
- temperatura;
- humedad;
- lluvia;
- nieve;
- indicador de día laboral.

```
formula_base <- bicicletas ~
  mes + hora_f + temperatura + humedad + lluvia + nieve + dia_laboral
```

6.1. Ajuste del modelo lineal

```
modelo_lineal <- lm(
  formula = formula_base,
  data = datos_operativos
)

summary(modelo_lineal)
```

```
##
## Call:
## lm(formula = formula_base, data = datos_operativos)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1635.90  -211.75    -7.12   204.20  1797.40
##
## Coefficients:
##                Estimate Std. Error t value             Pr(>|t|)
## (Intercept)    522.7535    27.0363  19.335 < 0.0000000000000002 ***
## mesFeb         -45.2417    18.9696  -2.385             0.017103 *
## mesMar         109.0517    21.8447   4.992  0.00000060911670792 ***
## mesAbr         196.4303    24.8220   7.914  0.00000000000000282 ***
## mesMay         360.8216    28.4215  12.695 < 0.0000000000000002 ***
## mesJun         498.8502    32.1077  15.537 < 0.0000000000000002 ***
## mesJul         157.2677    35.8270   4.390  0.00001149207652787 ***
## mesAgo           2.3649    36.5860   0.065             0.948463
## mesSep         357.0824    31.4197  11.365 < 0.0000000000000002 ***
## mesOct         459.1523    24.8610  18.469 < 0.0000000000000002 ***
## mesNov         303.3751    21.9419  13.826 < 0.0000000000000002 ***
## mesDic          65.8822    18.5156   3.558             0.000375 ***
## hora_f1       -104.0001    26.6078  -3.909  0.00009354649732485 ***
## hora_f2       -213.9025    26.6173  -8.036  0.00000000000000105 ***
## hora_f3       -298.6446    26.6298 -11.215 < 0.0000000000000002 ***
## hora_f4       -360.1788    26.6442 -13.518 < 0.0000000000000002 ***
## hora_f5       -346.3993    26.6670 -12.990 < 0.0000000000000002 ***
## hora_f6       -177.4363    26.6819  -6.650  0.00000000003112154 ***
## hora_f7        141.7943    26.6512   5.320  0.00000010621405150 ***
## hora_f8        529.0862    26.6076  19.885 < 0.0000000000000002 ***
## hora_f9         79.6822    26.6301   2.992             0.002778 **
## hora_f10      -133.4240    26.7598  -4.986  0.00000062874800784 ***
## hora_f11      -125.6825    27.0074  -4.654  0.00000331102793092 ***
## hora_f12       -75.2690    27.2655  -2.761             0.005782 **
## hora_f13       -74.9716    27.4967  -2.727             0.006413 **
## hora_f14       -83.2349    27.6126  -3.014             0.002583 **
## hora_f15       -14.0054    27.6795  -0.506             0.612882
## hora_f16        97.8358    27.5825   3.547             0.000392 ***
## hora_f17       355.2683    27.3648  12.983 < 0.0000000000000002 ***
## hora_f18       792.2457    27.1283  29.204 < 0.0000000000000002 ***
## hora_f19       518.7230    26.8678  19.307 < 0.0000000000000002 ***
## hora_f20       443.9558    26.7448  16.600 < 0.0000000000000002 ***
## hora_f21       440.0663    26.6644  16.504 < 0.0000000000000002 ***
## hora_f22       344.6789    26.6099  12.953 < 0.0000000000000002 ***
## hora_f23       108.9685    26.5905   4.098  0.00004206050298020 ***
## temperatura     25.3682     0.9679  26.210 < 0.0000000000000002 ***
## humedad         -7.8564     0.2357 -33.329 < 0.0000000000000002 ***
## lluvia         -67.0926     3.5632 -18.829 < 0.0000000000000002 ***
## nieve           31.1322     9.1976   3.385             0.000716 ***
## dia_laboralSí  102.2765     8.2332  12.423 < 0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 352.9 on 8425 degrees of freedom
## Multiple R-squared:  0.6995, Adjusted R-squared:  0.6981
## F-statistic: 502.9 on 39 and 8425 DF,  p-value: < 0.00000000000000022
```

Una interpretación directa de un coeficiente lineal es aditiva. Por ejemplo, manteniendo constantes las demás variables, un coeficiente de 20 para temperatura indicaría un aumento medio de 20 bicicletas por cada grado Celsius adicional.

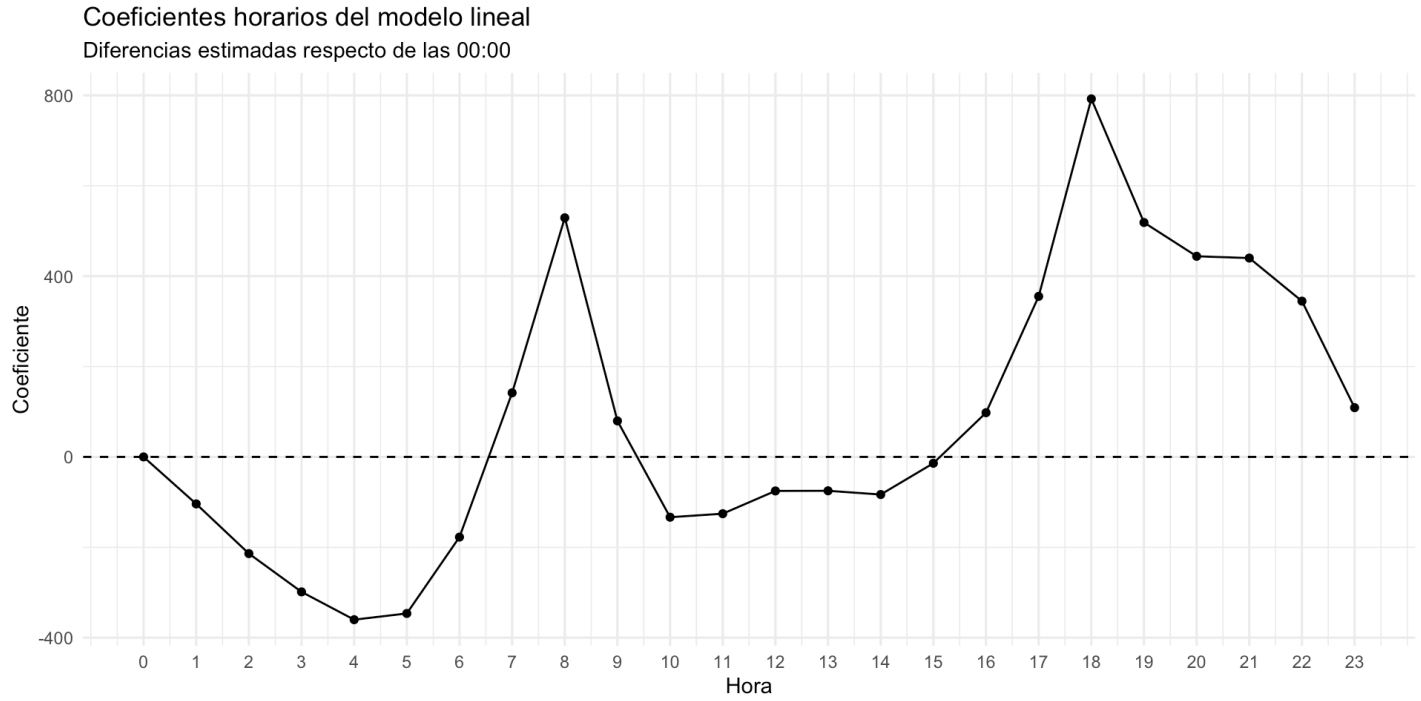
6.2. Coeficientes horarios

La hora 0 es la categoría de referencia. Los coeficientes restantes representan diferencias respecto de esa hora, manteniendo constantes las otras covariables.

```
coef_hora_lm <- tidy(modelo_lineal) |>
  filter(str_detect(term, "^hora_f")) |>
  mutate(hora = as.numeric(str_remove(term, "hora_f"))) |>
  select(hora, estimate)

coef_hora_lm <- bind_rows(
  tibble(hora = 0, estimate = 0),
  coef_hora_lm
) |>
  arrange(hora)

ggplot(coef_hora_lm, aes(x = hora, y = estimate)) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "Coeficientes horarios del modelo lineal",
    subtitle = "Diferencias estimadas respecto de las 00:00",
    x = "Hora",
    y = "Coeficiente"
  ) +
  theme_minimal()
```



6.3. Predicciones negativas

La regresión lineal no restringe las predicciones al conjunto de valores no negativos.

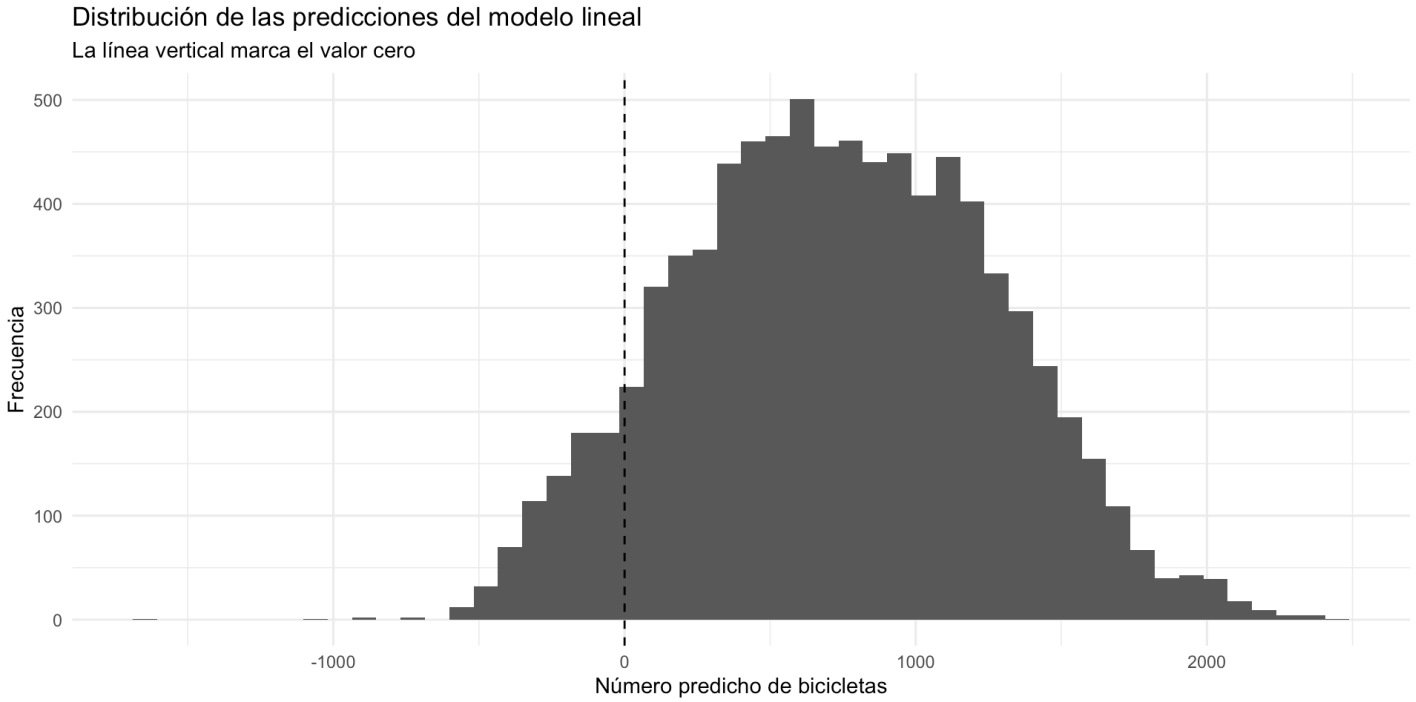
```
datos_operativos <- datos_operativos |>
  mutate(pred_lineal = predict(modelo_lineal, newdata = datos_operativos))

porcentaje_negativas <- mean(datos_operativos$pred_lineal < 0) * 100

cat(
  "Porcentaje de predicciones negativas:",
  round(porcentaje_negativas, 2),
  "%\n"
)
```

```
## Porcentaje de predicciones negativas: 9.11 %
```

```
ggplot(datos_operativos, aes(x = pred_lineal)) +
  geom_histogram(bins = 50) +
  geom_vline(xintercept = 0, linetype = 2) +
  labs(
    title = "Distribución de las predicciones del modelo lineal",
    subtitle = "La línea vertical marca el valor cero",
    x = "Número predicho de bicicletas",
    y = "Frecuencia"
  ) +
  theme_minimal()
```

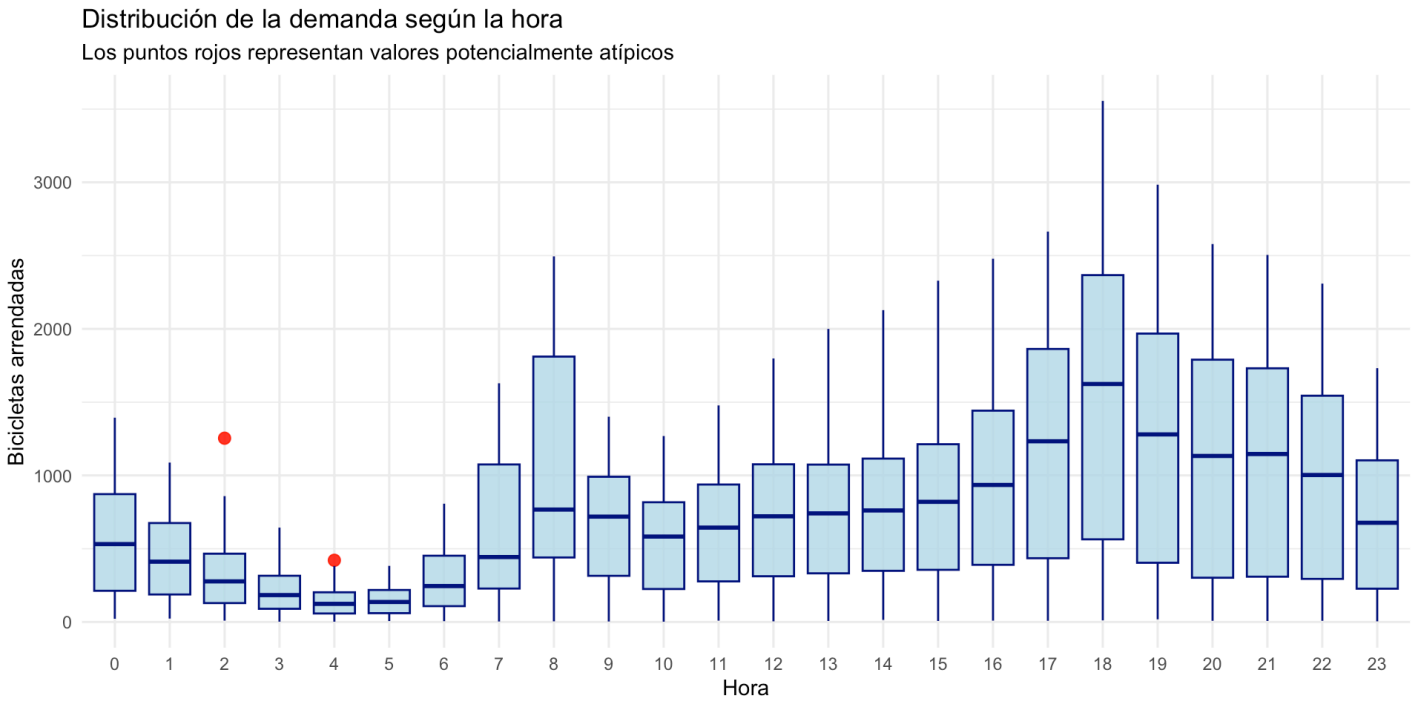


En esta base, el modelo lineal produce aproximadamente 9.1% de predicciones negativas. Una cantidad negativa de bicicletas carece de interpretación práctica. La estadística, a veces, puede ser muy creativa; el estacionamiento de bicicletas, bastante menos.

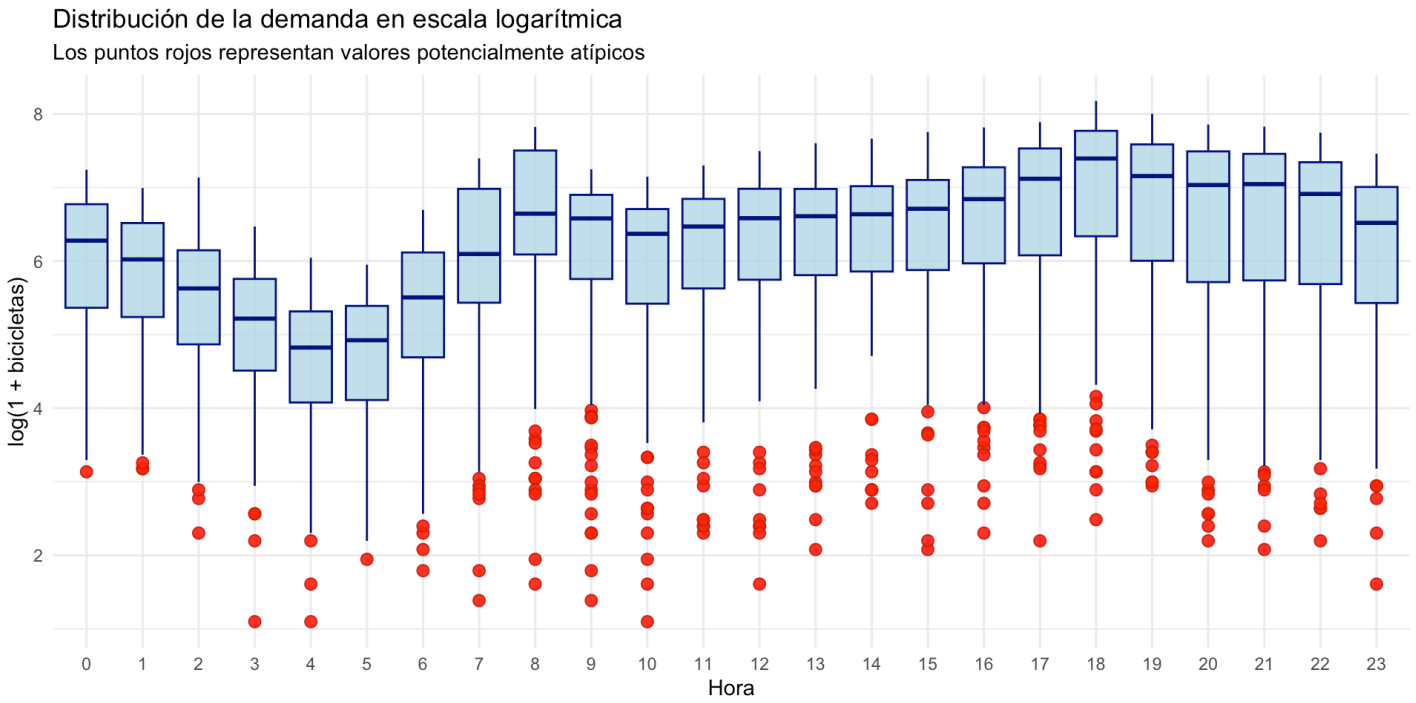
6.4. Relación entre media y variabilidad

La dispersión de la demanda cambia considerablemente según la hora. Cuando la demanda media es alta, también suele aumentar la variabilidad.

```
ggplot(datos_operativos, aes(x = hora_f, y = bicicletas)) +
  geom_boxplot(
    fill = "lightblue",
    colour = "navy",
    alpha = 0.75,
    outlier.colour = "red",
    outlier.fill = "red",
    outlier.shape = 21,
    outlier.size = 2.5,
    outlier.alpha = 0.9
  ) +
  labs(
    title = "Distribución de la demanda según la hora",
    subtitle = "Los puntos rojos representan valores potencialmente atípicos",
    x = "Hora",
    y = "Bicicletas arrendadas"
  ) +
  theme_minimal()
```




```
ggplot(
  datos_operativos,
  aes(x = hora_f, y = log1p(bicicletas))
) +
  geom_boxplot(
    fill = "lightblue",
    colour = "navy",
    alpha = 0.75,
    outlier.colour = "red3",
    outlier.fill = "red",
    outlier.shape = 21,
    outlier.size = 2.5,
    outlier.alpha = 0.9
  ) +
  labs(
    title = "Distribución de la demanda en escala logarítmica",
    subtitle = "Los puntos rojos representan valores potencialmente atípicos",
    x = "Hora",
    y = "log(1 + bicicletas)"
  ) +
  theme_minimal()
```



El modelo lineal supone, entre otras cosas, una varianza residual aproximadamente constante. Los gráficos sugieren que este supuesto no es razonable para la respuesta original.

7. Regresión de Poisson

Si Y_i representa el número de bicicletas arrendadas durante la hora i, el modelo de Poisson supone que

$$Y_i \mid \mathbf{x}_i \sim \text{Poisson}(\lambda_i),$$

donde

$$\log(\lambda_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}.$$

Por lo tanto,

$$\lambda_i = \mathbb{E}(Y_i \mid \mathbf{x}_i) = \exp(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}).$$

La función exponencial garantiza que la media estimada sea positiva.

7.1. Ajuste del modelo

```
modelo_poisson <- glm(
  formula = formula_base,
  family = poisson(link = "log"),
  data = datos_operativos
)

summary(modelo_poisson)
```

```
##
## Call:
## glm(formula = formula_base, family = poisson(link = "log"), data = datos_operativos)
##
## Coefficients:
##                   Estimate  Std. Error  z value             Pr(>|z|)
## (Intercept)    5.52905716  0.00379678 1456.248 < 0.0000000000000002 ***
## mesFeb         0.00845413  0.00366073    2.309             0.020921 *
## mesMar         0.64259446  0.00340394  188.780 < 0.0000000000000002 ***
## mesAbr         0.87318396  0.00360869  241.967 < 0.0000000000000002 ***
## mesMay         1.08314898  0.00391657  276.556 < 0.0000000000000002 ***
## mesJun         1.17024793  0.00428326  273.214 < 0.0000000000000002 ***
## mesJul         0.84681135  0.00479194  176.716 < 0.0000000000000002 ***
## mesAgo         0.69721095  0.00487247  143.092 < 0.0000000000000002 ***
## mesSep         1.06677718  0.00418545  254.878 < 0.0000000000000002 ***
## mesOct         1.18706995  0.00349218  339.922 < 0.0000000000000002 ***
## mesNov         1.00007715  0.00326023  306.750 < 0.0000000000000002 ***
## mesDic         0.24080226  0.00348380   69.120 < 0.0000000000000002 ***
## hora_f1       -0.22563910  0.00339006  -66.559 < 0.0000000000000002 ***
## hora_f2       -0.54376640  0.00376228 -144.531 < 0.0000000000000002 ***
## hora_f3       -0.90678963  0.00430767 -210.506 < 0.0000000000000002 ***
## hora_f4       -1.33534123  0.00507511 -263.116 < 0.0000000000000002 ***
## hora_f5       -1.28020628  0.00498081 -257.028 < 0.0000000000000002 ***
## hora_f6       -0.52504871  0.00382781 -137.167 < 0.0000000000000002 ***
## hora_f7        0.19256301  0.00310296   62.058 < 0.0000000000000002 ***
## hora_f8        0.66433430  0.00278727  238.346 < 0.0000000000000002 ***
## hora_f9        0.14793864  0.00305600   48.409 < 0.0000000000000002 ***
## hora_f10      -0.15283311  0.00322557  -47.382 < 0.0000000000000002 ***
## hora_f11      -0.09830374  0.00315488  -31.159 < 0.0000000000000002 ***
## hora_f12       0.01102340  0.00308678    3.571             0.000355 ***
## hora_f13       0.01254038  0.00308465    4.065            0.0000479 ***
## hora_f14       0.01274501  0.00308378    4.133            0.0000358 ***
## hora_f15       0.09587310  0.00304167   31.520 < 0.0000000000000002 ***
## hora_f16       0.21457741  0.00297020   72.243 < 0.0000000000000002 ***
## hora_f17       0.46133633  0.00285672  161.491 < 0.0000000000000002 ***
## hora_f18       0.78985636  0.00271326  291.110 < 0.0000000000000002 ***
## hora_f19       0.62146612  0.00275931  225.226 < 0.0000000000000002 ***
## hora_f20       0.56150441  0.00278278  201.778 < 0.0000000000000002 ***
## hora_f21       0.56858608  0.00278794  203.945 < 0.0000000000000002 ***
## hora_f22       0.47855181  0.00283699  168.683 < 0.0000000000000002 ***
## hora_f23       0.19108764  0.00302449   63.180 < 0.0000000000000002 ***
## temperatura    0.02700226  0.00011803  228.772 < 0.0000000000000002 ***
## humedad       -0.00706264  0.00002746 -257.157 < 0.0000000000000002 ***
## lluvia        -0.58177877  0.00226418 -256.949 < 0.0000000000000002 ***
## nieve         -0.12985626  0.00203192  -63.908 < 0.0000000000000002 ***
## dia_laboralSí  0.13704585  0.00088591  154.695 < 0.0000000000000002 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 4556385  on 8464  degrees of freedom
```

```
## Residual deviance:  829177  on 8425  degrees of freedom
## AIC: 896350
##
## Number of Fisher Scoring iterations: 7
```

7.2. Interpretación mediante razones de tasas

En un modelo de Poisson, $\exp(\beta_j)$ es una **razón de tasas** o un factor multiplicativo.

- Si $\exp(\beta_j) = 1$, no hay cambio en la media esperada.
- Si $\exp(\beta_j) > 1$, la media esperada aumenta.
- Si $\exp(\beta_j) < 1$, la media esperada disminuye.

Por ejemplo, si para temperatura se obtiene $\exp(\hat{\beta}) = 1.04$, un aumento de un grado Celsius se asocia con una demanda media aproximadamente 4% mayor, manteniendo constantes las demás variables.

```
efectos_poisson <- tidy(modelo_poisson) |>
  mutate(
    limite_inferior = estimate - 1.96 * std.error,
    limite_superior = estimate + 1.96 * std.error,
    razon_tasas = exp(estimate),
    rr_inferior = exp(limite_inferior),
    rr_superior = exp(limite_superior),
    cambio_porcentual = 100 * (razon_tasas - 1)
  ) |>
  filter(term %in% c(
    "temperatura",
    "humedad",
    "lluvia",
    "nieve",
    "dia_laboralSí"
  )) |>
  select(
    termino = term,
    coeficiente = estimate,
    razon_tasas,
    rr_inferior,
    rr_superior,
    cambio_porcentual,
    p_value = p.value
  )

kable(efectos_poisson, digits = 4)
```

termino	coeficiente	razon_tasas	rr_inferior	rr_superior	cambio_porcentual	p_value
temperatura	0.0270	1.0274	1.0271	1.0276	2.7370	0
humedad	-0.0071	0.9930	0.9929	0.9930	-0.7038	0
lluvia	-0.5818	0.5589	0.5564	0.5614	-44.1097	0

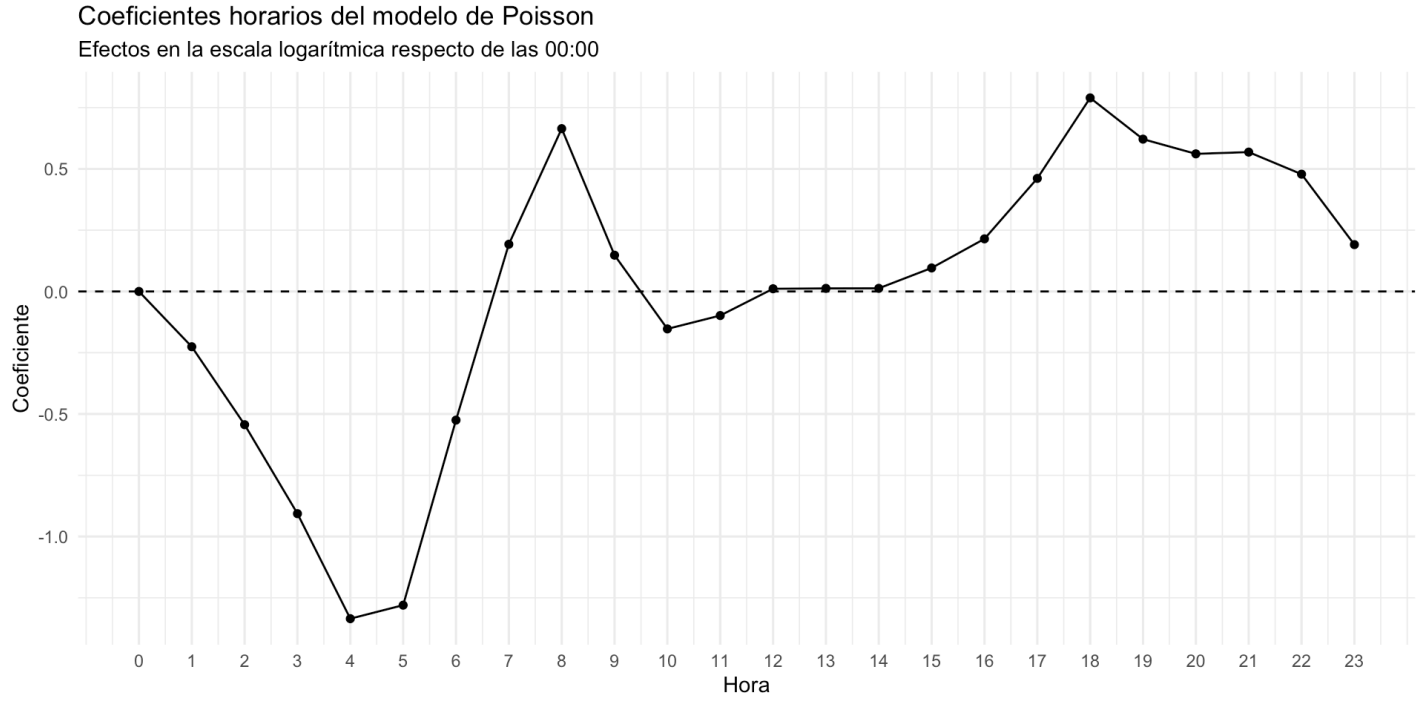
termino	coeficiente	razon_tasas	rr_inferior	rr_superior	cambio_porcentual	p_value
nieve	-0.1299	0.8782	0.8747	0.8817	-12.1778	0
dia_laboralSí	0.1370	1.1469	1.1449	1.1489	14.6881	0

7.3. Coeficientes horarios del modelo de Poisson

```
coef_hora_pois <- tidy(modelo_poisson) |>
  filter(str_detect(term, "^hora_f")) |>
  mutate(hora = as.numeric(str_remove(term, "hora_f"))) |>
  select(hora, estimate)

coef_hora_pois <- bind_rows(
  tibble(hora = 0, estimate = 0),
  coef_hora_pois
) |>
  arrange(hora)

ggplot(coef_hora_pois, aes(x = hora, y = estimate)) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "Coeficientes horarios del modelo de Poisson",
    subtitle = "Efectos en la escala logarítmica respecto de las 00:00",
    x = "Hora",
    y = "Coeficiente"
  ) +
  theme_minimal()
```



7.4. Predicciones del modelo de Poisson

```
datos_operativos <- datos_operativos |>
  mutate(
    pred_poisson = predict(
      modelo_poisson,
      newdata = datos_operativos,
      type = "response"
    )
  )

range(datos_operativos$pred_poisson)
```

```
## [1]    0.000001125941 3739.407087596574
```

Las predicciones de Poisson no son necesariamente enteras, porque representan una **media esperada**, pero siempre son no negativas.

7.5. Comparación visual entre valores observados y predichos

```
comparacion_hora <- datos_operativos |>
  group_by(hora) |>
  summarise(
    observado = mean(bicicletas, na.rm = TRUE),
    lineal = mean(pred_lineal, na.rm = TRUE),
    poisson = mean(pred_poisson, na.rm = TRUE),
    .groups = "drop"
  ) |>
  pivot_longer(
    cols = c(observado, lineal, poisson),
    names_to = "serie",
    values_to = "media"
  )

ggplot(comparacion_hora, aes(x = hora, y = media, color = serie)) +
  geom_line(linewidth = 1) +
  geom_point(size = 1.8) +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "Demanda media observada y predicha según la hora",
    x = "Hora",
    y = "Número medio de bicicletas",
    color = "Serie"
  ) +
  theme_minimal()
```



7.6. Medidas descriptivas del ajuste

Las siguientes medidas son calculadas sobre las mismas observaciones utilizadas para ajustar los modelos. Por lo tanto, describen el ajuste dentro de la muestra y no reemplazan una evaluación predictiva externa.

```
rmse <- function(real, predicho) {
  sqrt(mean((real - predicho)^2))
}

mae <- function(real, predicho) {
  mean(abs(real - predicho))
}

metricas_conteo <- tibble(
  modelo = c("Lineal", "Poisson"),
  RMSE = c(
    rmse(datos_operativos$bicicletas, datos_operativos$pred_lineal),
    rmse(datos_operativos$bicicletas, datos_operativos$pred_poisson)
  ),
  MAE = c(
    mae(datos_operativos$bicicletas, datos_operativos$pred_lineal),
    mae(datos_operativos$bicicletas, datos_operativos$pred_poisson)
  ),
  predicciones_negativas = c(
    sum(datos_operativos$pred_lineal < 0),
    sum(datos_operativos$pred_poisson < 0)
  )
)

kable(metricas_conteo, digits = 2)
```

modelo	RMSE	MAE	predicciones_negativas
Lineal	352.10	267.72	771
Poisson	279.68	187.32	0

No debe elegirse un modelo exclusivamente por una métrica predictiva. También deben considerarse la naturaleza de la respuesta, el soporte de las predicciones, los supuestos y la interpretación.

8. Diagnóstico de sobredispersión

La distribución de Poisson supone que, condicionalmente a las covariables,

$$\mathrm{Var}(Y_i \mid \mathbf{x}_i) = \mathbb{E}(Y_i \mid \mathbf{x}_i).$$

Una comprobación sencilla consiste en calcular

$$\widehat{\phi} = \frac{\sum_i r_{Pi}^2}{n - p},$$

donde r_{Pi} son los residuos de Pearson.

```
dispersion_poisson <- sum(
  residuals(modelo_poisson, type = "pearson")^2
) / df.residual(modelo_poisson)

dispersion_poisson
```

```
## [1] 2463442
```

El cociente de dispersión estimado es 2463442.36.

Como regla descriptiva:

- un valor cercano a 1 es compatible con la varianza de Poisson;
- un valor claramente mayor que 1 sugiere sobredispersión;
- un valor menor que 1 sugiere subdispersión.

Esta regla no sustituye un diagnóstico completo, pero sirve como alerta inicial.

8.1. Modelo quasi-Poisson

El modelo quasi-Poisson conserva la media y el enlace logarítmico del modelo de Poisson, pero permite

$$\mathrm{Var}(Y_i \mid \mathbf{x}_i) = \phi\, \mathbb{E}(Y_i \mid \mathbf{x}_i).$$

Los coeficientes estimados suelen coincidir con los de Poisson, mientras que los errores estándar se corrigen por la dispersión estimada.

```
modelo_quasipoisson <- glm(
  formula = formula_base,
  family = quasipoisson(link = "log"),
  data = datos_operativos
)

summary(modelo_quasipoisson)
```

```
##
## Call:
## glm(formula = formula_base, family = quasipoisson(link = "log"),
##     data = datos_operativos)
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.529057   5.959208   0.928    0.354
## mesFeb         0.008454   5.745672   0.001    0.999
## mesMar         0.642594   5.342619   0.120    0.904
## mesAbr         0.873184   5.663991   0.154    0.877
## mesMay         1.083149   6.147214   0.176    0.860
## mesJun         1.170248   6.722761   0.174    0.862
## mesJul         0.846811   7.521147   0.113    0.910
## mesAgo         0.697211   7.647549   0.091    0.927
## mesSep         1.066777   6.569232   0.162    0.871
## mesOct         1.187070   5.481128   0.217    0.829
## mesNov         1.000077   5.117070   0.195    0.845
## mesDic         0.240802   5.467975   0.044    0.965
## hora_f1       -0.225639   5.320835  -0.042    0.966
## hora_f2       -0.543766   5.905048  -0.092    0.927
## hora_f3       -0.906790   6.761065  -0.134    0.893
## hora_f4       -1.335341   7.965592  -0.168    0.867
## hora_f5       -1.280206   7.817590  -0.164    0.870
## hora_f6       -0.525049   6.007911  -0.087    0.930
## hora_f7        0.192563   4.870231   0.040    0.968
## hora_f8        0.664334   4.374734   0.152    0.879
## hora_f9        0.147939   4.796526   0.031    0.975
## hora_f10      -0.152833   5.062669  -0.030    0.976
## hora_f11      -0.098304   4.951708  -0.020    0.984
## hora_f12       0.011023   4.844822   0.002    0.998
## hora_f13       0.012540   4.841483   0.003    0.998
## hora_f14       0.012745   4.840126   0.003    0.998
## hora_f15       0.095873   4.774026   0.020    0.984
## hora_f16       0.214577   4.661849   0.046    0.963
## hora_f17       0.461336   4.483745   0.103    0.918
## hora_f18       0.789856   4.258573   0.185    0.853
## hora_f19       0.621466   4.330844   0.143    0.886
## hora_f20       0.561504   4.367688   0.129    0.898
## hora_f21       0.568586   4.375784   0.130    0.897
## hora_f22       0.478552   4.452774   0.107    0.914
## hora_f23       0.191088   4.747060   0.040    0.968
## temperatura    0.027002   0.185255   0.146    0.884
## humedad       -0.007063   0.043106  -0.164    0.870
## lluvia        -0.581779   3.553729  -0.164    0.870
## nieve         -0.129856   3.189181  -0.041    0.968
## dia_laboralSí  0.137046   1.390470   0.099    0.921
##
## (Dispersion parameter for quasipoisson family taken to be 2463460)
##
##     Null deviance: 4556385  on 8464  degrees of freedom
## Residual deviance:  829177  on 8425  degrees of freedom
```

```
## AIC: NA
##
## Number of Fisher Scoring iterations: 7
```

8.2. Comparación de errores estándar

```
comparacion_errores <- tidy(modelo_poisson) |>
  select(term, beta_poisson = estimate, se_poisson = std.error) |>
  left_join(
    tidy(modelo_quasipoisson) |>
      select(term, beta_quasi = estimate, se_quasi = std.error),
    by = "term"
  ) |>
  filter(term %in% c(
    "temperatura",
    "humedad",
    "lluvia",
    "nieve",
    "dia_laboralSí"
  ))

kable(comparacion_errores, digits = 5)
```

term	beta_poisson	se_poisson	beta_quasi	se_quasi
temperatura	0.02700	0.00012	0.02700	0.18525
humedad	-0.00706	0.00003	-0.00706	0.04311
lluvia	-0.58178	0.00226	-0.58178	3.55373
nieve	-0.12986	0.00203	-0.12986	3.18918
dia_laboralSí	0.13705	0.00089	0.13705	1.39047

El quasi-Poisson es especialmente útil para corregir la inferencia cuando existe sobredispersión. Sin embargo, como no especifica una distribución de probabilidad completa, no dispone de una verosimilitud ordinaria y no debe compararse mediante AIC con el modelo de Poisson.

9. Clasificación sencilla mediante regresión logística

Ahora formularemos un problema diferente. En lugar de predecir el número exacto de bicicletas, clasificaremos cada hora como:

- **Alta demanda**: el conteo supera el percentil 75 observado en el conjunto de entrenamiento;
- **Demanda normal**: el conteo no supera ese umbral.

La definición es pedagógica. En una aplicación real, el umbral debería establecerse con criterios operativos, como la capacidad disponible o el costo de quedarse sin bicicletas.

9.1. División temporal en entrenamiento y prueba

Como las observaciones están ordenadas en el tiempo, utilizamos las primeras 80% para entrenar y las últimas 20% para evaluar. Esto es más exigente y más realista que mezclar aleatoriamente observaciones pasadas y futuras.

```
n_total <- nrow(datos_operativos)
indice_corte <- floor(0.80 * n_total)

entrenamiento <- datos_operativos |>
  slice(1:indice_corte)

prueba <- datos_operativos |>
  slice((indice_corte + 1):n_total)

range(entrenamiento$fecha)
```

```
## [1] "2017-12-01" "2018-09-11"
```

```
range(prueba$fecha)
```

```
## [1] "2018-09-11" "2018-11-30"
```

El umbral de alta demanda se calcula **solo con entrenamiento**, evitando utilizar información de la muestra de prueba.

```
umbral_alta <- as.numeric(
  quantile(entrenamiento$bicicletas, probs = 0.75)
)

entrenamiento <- entrenamiento |>
  mutate(
    alta_demanda = factor(
      if_else(bicicletas > umbral_alta, "Alta", "Normal"),
      levels = c("Normal", "Alta")
    )
  )

prueba <- prueba |>
  mutate(
    alta_demanda = factor(
      if_else(bicicletas > umbral_alta, "Alta", "Normal"),
      levels = c("Normal", "Alta")
    )
  )

cat("Umbral de alta demanda:", umbral_alta, "bicicletas\n")
```

```
## Umbral de alta demanda: 1027 bicicletas
```

```
prop.table(table(entrenamiento$alta_demanda))
```

```
##
##    Normal      Alta
## 0.7502953 0.2497047
```

```
prop.table(table(prueba$alta_demanda))
```

```
##
##    Normal      Alta
## 0.6284702 0.3715298
```

9.2. ¿Por qué usar seno y coseno para hora y mes?

La hora y el mes son variables cíclicas. Las 23:00 están cerca de las 00:00, y diciembre está cerca de enero. Para conservar esta estructura utilizamos:

$$\sin\left(\frac{2\pi h}{24}\right), \qquad \cos\left(\frac{2\pi h}{24}\right),$$

junto con expresiones análogas para el mes.

Además, esta codificación permite predecir meses futuros aunque algunos meses no aparezcan como categorías observadas en el conjunto de entrenamiento.

9.3. Ajuste del modelo logístico

Sea $Y_i = 1$ si la hora tiene alta demanda y $Y_i = 0$ en otro caso. El modelo logístico es

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip},$$

donde $p_i = P(Y_i = 1 \mid \mathbf{x}_i)$.

```
modelo_logistico <- glm(
  alta_demanda ~
    hora_sen + hora_cos +
    mes_sen + mes_cos +
    temperatura + humedad + lluvia +
    dia_laboral,
  family = binomial(link = "logit"),
  data = entrenamiento
)

summary(modelo_logistico)
```

```
##
## Call:
## glm(formula = alta_demanda ~ hora_sen + hora_cos + mes_sen +
##     mes_cos + temperatura + humedad + lluvia + dia_laboral, family = binomial(link =
"logit"),
##     data = entrenamiento)
##
## Coefficients:
##                Estimate Std. Error z value             Pr(>|z|)
## (Intercept)   -1.848785   0.225495  -8.199 0.000000000000000243 ***
## hora_sen      -1.729616   0.071738 -24.110 < 0.0000000000000002 ***
## hora_cos       0.111168   0.060298   1.844             0.065238 .
## mes_sen       -0.236235   0.107718  -2.193             0.028301 *
## mes_cos       -2.656369   0.144856 -18.338 < 0.0000000000000002 ***
## temperatura    0.028682   0.009818   2.922             0.003483 **
## humedad       -0.021149   0.002521  -8.390 < 0.0000000000000002 ***
## lluvia        -2.945282   0.450233  -6.542 0.000000000060831042 ***
## dia_laboralSí  0.296098   0.084203   3.516             0.000437 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 7611.9  on 6771  degrees of freedom
## Residual deviance: 4236.8  on 6763  degrees of freedom
## AIC: 4254.8
##
## Number of Fisher Scoring iterations: 9
```

9.4. Odds ratios

En regresión logística, $\exp(\beta_j)$ representa un **odds ratio**.

```
odds_ratios <- tidy(modelo_logistico) |>
  mutate(
    limite_inferior = estimate - 1.96 * std.error,
    limite_superior = estimate + 1.96 * std.error,
    OR = exp(estimate),
    OR_inferior = exp(limite_inferior),
    OR_superior = exp(limite_superior)
  ) |>
  select(
    termino = term,
    coeficiente = estimate,
    OR,
    OR_inferior,
    OR_superior,
    p_value = p.value
  )

kable(odds_ratios, digits = 4)
```

termino	coeficiente	OR	OR_inferior	OR_superior	p_value
(Intercept)	-1.8488	0.1574	0.1012	0.2449	0.0000
hora_sen	-1.7296	0.1774	0.1541	0.2041	0.0000
hora_cos	0.1112	1.1176	0.9930	1.2578	0.0652
mes_sen	-0.2362	0.7896	0.6393	0.9752	0.0283
mes_cos	-2.6564	0.0702	0.0529	0.0933	0.0000
temperatura	0.0287	1.0291	1.0095	1.0491	0.0035
humedad	-0.0211	0.9791	0.9742	0.9839	0.0000
lluvia	-2.9453	0.0526	0.0218	0.1271	0.0000
dia_laboralSí	0.2961	1.3446	1.1400	1.5859	0.0004

Los efectos de `hora_sen`, `hora_cos`, `mes_sen` y `mes_cos` deben interpretarse en conjunto, porque representan ciclos completos y no incrementos unitarios ordinarios.

9.5. Probabilidades y clases predichas

Utilizamos inicialmente el punto de corte 0.50:

$$\widehat{Y}_i = \begin{cases} 1, & \widehat{p}_i \geq 0.50, \\ 0, & \widehat{p}_i < 0.50. \end{cases}$$

```
punto_corte <- 0.50

prueba <- prueba |>
  mutate(
    prob_alta = predict(
      modelo_logistico,
      newdata = prueba,
      type = "response"
    ),
    clase_predicha = factor(
      if_else(prob_alta >= punto_corte, "Alta", "Normal"),
      levels = c("Normal", "Alta")
    )
  )

prueba |>
  select(
    fecha,
    hora,
    bicicletas,
    alta_demanda,
    prob_alta,
    clase_predicha
  ) |>
  head(10) |>
  kable(digits = 3)
```

fecha	hora	bicicletas	alta_demanda	prob_alta	clase_predicha
2018-09-11	4	193	Normal	0.034	Normal
2018-09-11	5	258	Normal	0.026	Normal
2018-09-11	6	678	Normal	0.024	Normal
2018-09-11	7	1556	Alta	0.024	Normal
2018-09-11	8	2369	Alta	0.030	Normal
2018-09-11	9	1234	Alta	0.044	Normal
2018-09-11	10	922	Normal	0.072	Normal
2018-09-11	11	1012	Normal	0.115	Normal
2018-09-11	12	1192	Alta	0.172	Normal
2018-09-11	13	1102	Alta	0.253	Normal

10. Evaluación del clasificador

10.1. Matriz de confusión

```
matriz_confusion <- table(
  Real = prueba$alta_demanda,
  Predicho = prueba$clase_predicha
)

kable(addmargins(matriz_confusion))
```

	Normal	Alta	Sum
Normal	1064	0	1064
Alta	609	20	629
Sum	1673	20	1693

La matriz contiene:

- verdaderos negativos: demanda normal clasificada como normal;
- falsos positivos: demanda normal clasificada como alta;
- falsos negativos: demanda alta clasificada como normal;
- verdaderos positivos: demanda alta clasificada como alta.

10.2. Métricas de clasificación

```
division_segura <- function(numerador, denominador) {
  ifelse(denominador == 0, NA_real_, numerador / denominador)
}

VN <- matriz_confusion["Normal", "Normal"]
FP <- matriz_confusion["Normal", "Alta"]
FN <- matriz_confusion["Alta", "Normal"]
VP <- matriz_confusion["Alta", "Alta"]

metricas_clasificacion <- tibble(
  metrica = c(
    "Exactitud",
    "Sensibilidad",
    "Especificidad",
    "Precisión",
    "Valor F1"
  ),
  valor = c(
    division_segura(VP + VN, VP + VN + FP + FN),
    division_segura(VP, VP + FN),
    division_segura(VN, VN + FP),
    division_segura(VP, VP + FP),
    division_segura(
      2 * VP,
      2 * VP + FP + FN
    )
  )
)

exactitud_base <- max(prop.table(table(prueba$alta_demanda)))

kable(metricas_clasificacion, digits = 3)
```

metrica	valor
Exactitud	0.640
Sensibilidad	0.032
Especificidad	1.000
Precisión	1.000
Valor F1	0.062

```
cat("Exactitud del clasificador mayoritario:", round(exactitud_base, 3), "\n")
```

```
## Exactitud del clasificador mayoritario: 0.628
```

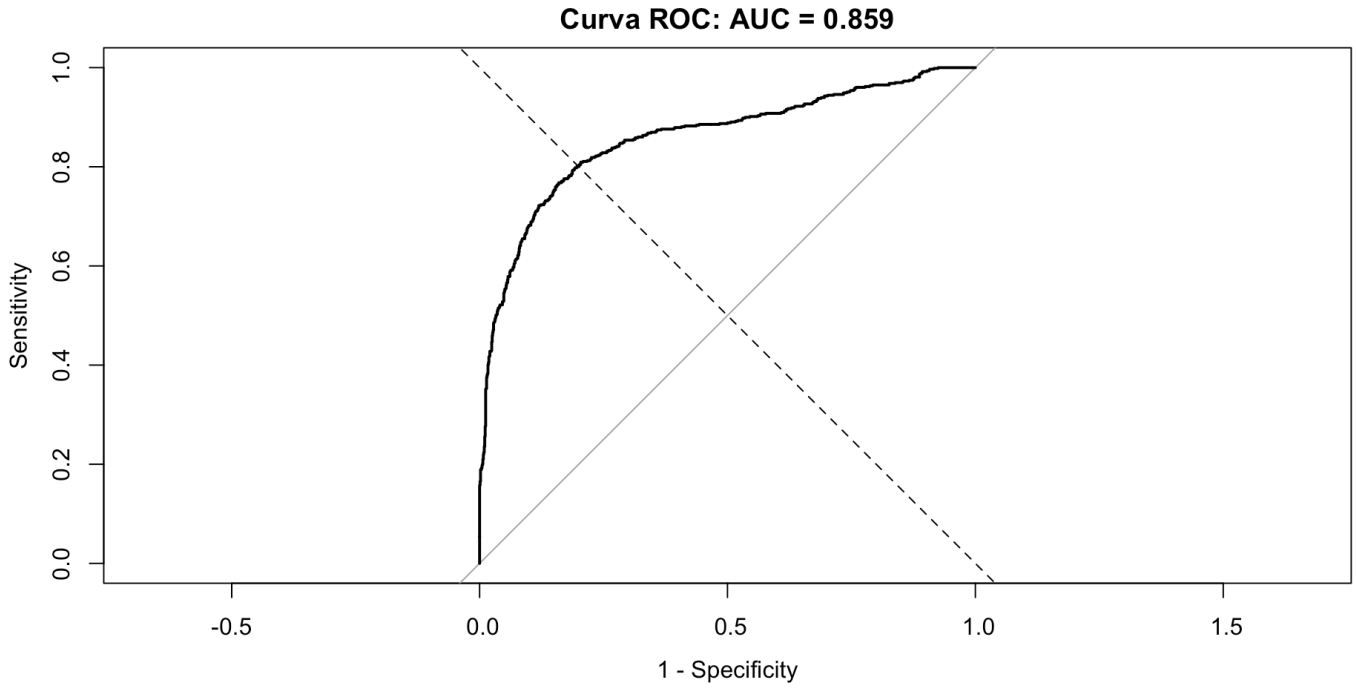
La exactitud debe compararse con el clasificador mayoritario. Si la clase de alta demanda es poco frecuente, un modelo que siempre prediga “Normal” puede obtener una exactitud aparentemente buena sin ser útil.

10.3. Curva ROC y AUC

```
roc_logistica <- pROC::roc(
  response = prueba$alta_demanda,
  predictor = prueba$prob_alta,
  levels = c("Normal", "Alta"),
  direction = "<",
  quiet = TRUE
)

auc_logistica <- as.numeric(pROC::auc(roc_logistica))

plot(
  roc_logistica,
  main = paste0("Curva ROC: AUC = ", round(auc_logistica, 3)),
  legacy.axes = TRUE
)
abline(a = 0, b = 1, lty = 2)
```



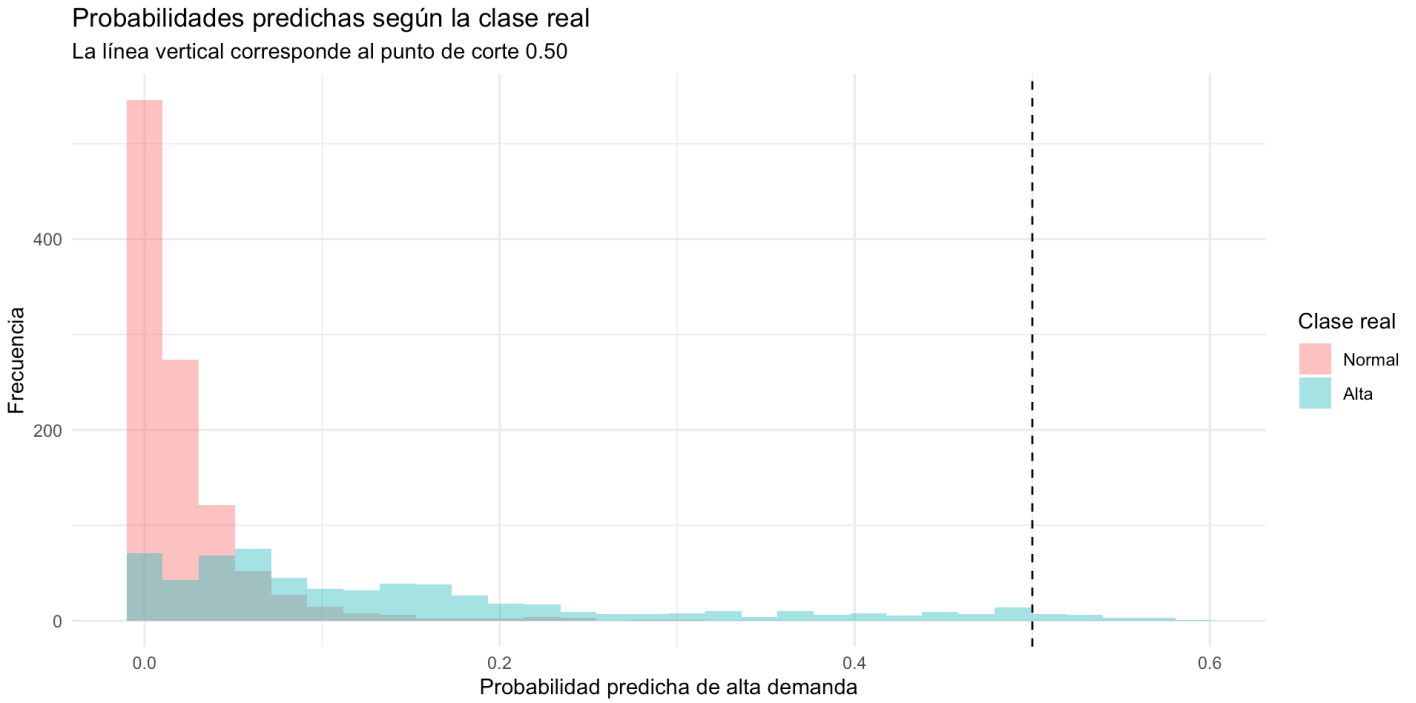

```
auc_logistica
```

```
## [1] 0.859163
```

El AUC resume la capacidad del modelo para asignar probabilidades mayores a las observaciones de alta demanda que a las de demanda normal. Un AUC cercano a 0.5 corresponde a una discriminación similar al azar, mientras que valores mayores indican una mejor capacidad discriminante.

10.4. Distribución de probabilidades predichas

```
ggplot(prueba, aes(x = prob_alta, fill = alta_demanda)) +
  geom_histogram(
    position = "identity",
    alpha = 0.45,
    bins = 30
  ) +
  geom_vline(xintercept = punto_corte, linetype = 2) +
  labs(
    title = "Probabilidades predichas según la clase real",
    subtitle = "La línea vertical corresponde al punto de corte 0.50",
    x = "Probabilidad predicha de alta demanda",
    y = "Frecuencia",
    fill = "Clase real"
  ) +
  theme_minimal()
```



11. Comparación conceptual de los modelos

Modelo	Tipo de respuesta	Distribución	Función de enlace	Interpretación principal
Lineal	Continua	Normal	Identidad	Cambio aditivo en la media
Poisson	Conteo	Poisson	Logaritmo	Razón de tasas $\exp(\beta)$

Modelo	Tipo de respuesta	Distribución	Función de enlace	Interpretación principal
Quasi-Poisson	Conteo sobredisperso	Cuasi-verosimilitud	Logaritmo	Razón de tasas con errores estándar corregidos
Logístico	Binaria	Bernoulli/binomial	Logit	Odds ratio $\exp(\beta)$

Los modelos comparten la estructura general

$$g\{\mathbb{E}(Y \mid X)\} = X\beta,$$

pero utilizan distintas distribuciones y funciones de enlace según la naturaleza de la variable respuesta.